

گزارش فاز اول و دوم پروژه برنامه‌سازی وب

سامانه پخش و مدیریت موسیقی

۱. نقش اعضای گروه و فعالیت‌های انجام‌شده

پارسا حاجی قاسمی

فاز اول: روی تکمیل و بازبینی قسمت‌های مختلف رابط کاربری کار کردم. صفحات مدیریتی، اعلان‌ها و تنظیمات را پیاده‌سازی و اصلاح کردم و حالت واکنش‌گرای صفحات را برای نمایشگرهای مختلف کامل کردم. همچنین برای بخش‌های فرانت‌اند آزمون نوشتن و هماهنگی اجزای مشترک رابط کاربری را بررسی کردم.

فاز دوم: رابط کاربری را به سرویس‌های واقعی بک‌اند متصل کردم. در این بخش لایه ارتباط با API، مدیریت توکن‌های احراز هویت، تبدیل ساختار داده‌های فرانت‌اند و بک‌اند و مدیریت خطاهای شبکه را تکمیل کردم. پخش‌کننده موسیقی را با قابلیت‌هایی مثل صف پخش، تکرار، پخش تصادفی، انتخاب کیفیت، نوار پیشرفت و Crossfade توسعه دادم و خطاهای تعویض یا شروع دوباره آهنگ را برطرف کردم. در پایان، اجرای یکپارچه دو بخش را با Docker Compose آماده کردم و عملکرد آن‌ها را با آزمون‌های فرانت‌اند و بک‌اند بررسی کردم.

سپهر رمضانی

فاز اول: ساختار اولیه پروژه را با Next.js و App Router راه‌اندازی کردم. داده‌های نمونه، توابع ذخیره‌سازی در localStorage و لایه انتزاعی mockApi را برای شبیه‌سازی ارتباط با سرور پیاده‌سازی کردم. سپس صفحات ورود و ثبت‌نام کاربر عادی و هنرمند را ساختم و جریان اولیه ورود کاربران به برنامه را تکمیل کردم.

فاز دوم: مدل‌های اصلی بک‌اند و اندپوینت‌های مربوط به احراز هویت، کاربران، هنرمندان، آهنگ‌ها، آلبوم‌ها و پلی‌لیست‌ها را پیاده‌سازی کردم. در این بخش روابط داده‌ها و مسیرهای لازم برای دریافت، ایجاد و ویرایش اطلاعات را آماده کردم تا فرانت‌اند بتواند به‌جای داده‌های نمونه از منابع واقعی استفاده کند.

سارینا فرزادنسب

فاز اول: بخش‌های ۶ تا ۹ را پیاده‌سازی کردم. این بخش‌ها شامل اعلانات، پلی‌لیست‌ها، صفحات آلبوم‌ها و تک‌آهنگ‌ها و پخش‌کننده موسیقی بودند. در این صفحات، نمایش و مدیریت فهرست آهنگ‌ها و رفتارهای اصلی پخش مانند صف، تکرار، پخش تصادفی و نوار پیشرفت را تکمیل کردم.

فاز دوم: بک‌اند سامانه موسیقی را با Django و REST API راه‌اندازی کردم و مدل‌ها، مهاجرت‌های پایگاه داده، احراز هویت JWT، ثبت‌نام، پروفایل و سطح دسترسی کاربران را پیاده‌سازی کردم. مدیریت هنرمندان و آثار، آپلود موسیقی و آلبوم، پلی‌لیست، صف پخش، ثبت آمار استریم و تأیید هنرمندان را توسعه دادم. همچنین سیستم تیکت، حسابرسی مالی، قیمت‌گذاری اشتراک‌ها، اعلانات و تنظیمات کاربران را تکمیل کردم. در پایان، پرداخت آزمایشی، مدیریت چرخه اشتراک و گزارش‌های تجمیعی مالی را همراه با مستندات و تست‌های مربوطه پیاده‌سازی کردم.

۲. ساختار و پیاده‌سازی فاز اول

در فاز اول تمرکز ما روی ساخت فرانت‌اند بود. برای این کار از React، Next.js، App Router و Tailwind CSS استفاده کردیم. مسیرها را بر اساس صفحه‌های برنامه جدا کردیم و قسمت‌هایی که در چند صفحه استفاده می‌شوند، مثل نوار کناری، پوسته اصلی، کارت‌ها و پخش‌کننده را داخل پوشه components قرار دادیم. وضعیت کاربر و پخش موسیقی هم در useContext و PlayerContext نگهداری می‌شود.

چون در شروع کار بک‌اند آماده نبود، داده‌های نمونه را در `seedData.js` قرار دادیم و عملیات موقت را با `mockApi.js` انجام دادیم. به این شکل توانستیم صفحه‌های ورود و ثبت‌نام، خانه، کتابخانه، پروفایل، پلی‌لیست، اعلان‌ها، تنظیمات و داشبوردهای هنرمند و مدیر را زودتر کامل کنیم. رابط کاربری با توجه به نقش کاربر تغییر می‌کند و صفحه‌ها را برای موبایل، تبلت و دسکتاپ طراحی کردیم.

برای اینکه صفحه‌ها مستقیماً به شکل ذخیره‌سازی داده وابسته نباشند، همه عملیات خواندن و تغییر داده را پشت توابع مشخص قرار دادیم. این روش باعث شد هنگام آماده شدن بک‌اند، بتوانیم پیاده‌سازی ماک را با درخواست‌های واقعی جایگزین کنیم، بدون آنکه منطق اصلی کامپوننت‌ها دوباره نوشته شود. در طراحی هر صفحه نیز حالت‌های بارگذاری، نبود داده، خطا و تفاوت دسترسی نقش‌های مختلف را در نظر گرفتیم تا مسیر کاربر در شرایط گوناگون مشخص و قابل پیش‌بینی باشد.

۳. ساختار و پیاده‌سازی فاز دوم

در فاز دوم بک‌اند را با Django و Django REST Framework ساختیم و بعد آن را به فرانت‌اند وصل کردیم. برای اینکه کد بک‌اند مرتب‌تر باشد، آن را به چهار برنامه `users`، `music`، `payments` و `support` تقسیم کردیم. مدل‌ها، سریالایزرها، دسترسی‌ها، سرویس‌ها، مسیرها و آزمون‌های هر بخش در همان برنامه قرار گرفته‌اند.

احراز هویت با توکن‌های JWT انجام می‌شود و دسترسی‌ها بر اساس نقش کاربر و سطح اشتراک او کنترل می‌شوند. در فرانت‌اند، `apiClient.js` وظیفه ارسال درخواست، اضافه کردن توکن، تازه‌سازی توکن منقضی‌شده و یکسان کردن خطاها را دارد. فایل `api.js` هم داده‌های `snake_case` بک‌اند را به شکل مورد نیاز کامپوننت‌ها تبدیل می‌کند. در نتیجه، بعد از اتصال دو بخش، داده‌های ماک از مسیر اصلی برنامه کنار رفتند و اطلاعات از پایگاه داده و فایل‌های واقعی خوانده شدند.

در سمت سرور، سریالایزرها علاوه بر تبدیل داده‌ها، ورودی‌ها را اعتبارسنجی می‌کنند و کلاس‌های دسترسی مالکیت منبع، نقش کاربر و محدودیت اشتراک را پیش از اجرای عملیات بررسی می‌کنند. منطق‌هایی مثل ثبت استریم، مدیریت چرخه اشتراک، پرداخت و محاسبه گزارش‌های مالی در بک‌اند انجام می‌شوند تا نتیجه برای همه کلاینت‌ها یکسان باشد. مسیرهای آپلود نیز فایل‌های موسیقی و تصاویر را جدا از داده‌های متنی مدیریت می‌کنند و پاسخ‌های API با ساختار مشخص به فرانت‌اند برمی‌گردند.

طبق توضیحات README، فرانت‌اند و بک‌اند را با Docker Compose اجرا می‌کنیم. فرانت‌اند روی پورت ۳۰۰۰ و API روی پورت ۸۰۰۰ در دسترس است. داده‌ها و فایل‌های آپلودشده هم در یک حجم پایدار نگهداری می‌شوند تا با خاموش شدن کانتینرها از بین نروند.

۴. نگهداری پذیری و توسعه پذیری

برای اینکه تغییر دادن پروژه در ادامه راحت‌تر باشد، صفحه‌ها، کامپونت‌ها، وضعیت برنامه و کدهای مربوط به ارتباط شبکه را از هم جدا کردیم. به همین دلیل تغییر ظاهر یک صفحه مستقیماً روی منطق دریافت داده اثر نمی‌گذارد. در بک‌اند هم هر حوزه را در یک برنامه جدا قرار دادیم تا فایل‌ها بیش از حد بزرگ نشوند و مسئولیت بخش‌ها با هم مخلوط نشود. قوانین مهمی مثل سطح دسترسی، محدودیت اشتراک، شمارش پخش و بررسی پرداخت را در سرور قرار دادیم تا همه کلاينت‌ها رفتار یکسانی داشته باشند.

نام‌گذاری منظم، متغیرهای محیطی، مهاجرت‌های پایگاه داده، سرویس‌های جدا، محدودیت‌های دیتابیس و آزمون‌های خودکار باعث می‌شوند اضافه کردن قابلیت جدید یا تغییر بخش‌های قبلی کم‌ریسک‌تر باشد. داکر هم کمک می‌کند همه اعضای گروه پروژه را در یک محیط مشابه و با یک فرمان مشخص اجرا کنند.

برای جلوگیری از شکستن بخش‌های دیگر، قرارداد ورودی و خروجی اندپوینت‌ها را ثابت نگه داشتیم و تغییرات ساختار پایگاه داده را فقط از طریق مهاجرت‌ها اعمال کردیم. آزمون‌های واحد منطق هر بخش را جداگانه بررسی می‌کنند و آزمون‌های یکپارچه مسیرهای مهمی مانند ورود، دریافت داده، پخش موسیقی و خرید اشتراک را از ابتدا تا انتها می‌سنجند. مستندات اجرا و متغیرهای لازم نیز در کنار پروژه نگهداری می‌شوند تا راه‌اندازی محیط توسعه برای اعضای گروه تکرارپذیر باشد.

۵. مدل‌های بک‌اند و روابط آن‌ها

در بک‌اند، مدل CustomUser اطلاعات اصلی کاربر، نقش و سطح اشتراک او را نگه می‌دارد. مدل UserSettings با یک رابطه یک‌به‌یک به کاربر متصل است و تنظیمات او را ذخیره می‌کند. پروفایل Artist هم به صورت یک‌به‌یک به کاربر هنرمند وصل شده است. هر هنرمند می‌تواند چند آلبوم و آهنگ داشته باشد و هر آهنگ هم می‌تواند عضو یک آلبوم باشد یا چند هنرمند مهمان داشته باشد.

هر Playlist متعلق به یک کاربر است. برای رابطه پلی‌لیست و آهنگ از مدل واسط PlaylistTrack استفاده کردیم تا ترتیب آهنگ‌ها مشخص باشد و یک آهنگ دوبار داخل یک پلی‌لیست قرار نگیرد. هر کاربر یک PlaybackQueue دارد و آهنگ‌های مرتب‌شده صف در QueueItem ذخیره می‌شوند. مدل StreamEvent هم هر بار پخش شدن یک آهنگ توسط کاربر را ثبت می‌کند.

مدل‌های Transaction و UserSubscription تاریخچه پرداخت و دوره اشتراک کاربر را نگهداری می‌کنند و هر اشتراک پرداختی به یک تراکنش موفق وصل است. اعلان‌ها و تیکت‌ها هم به کاربر مربوط هستند و هر پاسخ تیکت به تیکت اصلی و نویسنده پاسخ متصل می‌شود. این روابط کمک می‌کنند مالک هر داده و نحوه کنترل دسترسی به آن مشخص باشد.

قیمت‌های قابل تغییر اشتراک در مدل SubscriptionPrice نگهداری می‌شوند تا تغییر آن‌ها نیازی به ویرایش کد نداشته باشد. مدل ArtistMonthlyStatement نیز آمار ماهانه هنرمند، مبلغ محاسبه‌شده و وضعیت تسویه را ثبت می‌کند. برای رابطه‌های حساس از محدودیت یکتایی و مدل‌های واسط استفاده کردیم تا داده‌های تکراری یا ترتیب نامعتبر ایجاد نشوند و حذف یک رکورد، رکوردهای وابسته را در وضعیت ناسازگار باقی نگذارد.

۶. نقش هوش مصنوعی در توسعه

در قسمت‌های مختلف پروژه برای پیدا کردن راه‌حل، رفع خطا، نوشتن آزمون، بازبینی ساختار و رعایت best practice از هوش مصنوعی کمک گرفتیم. البته کدهای پیشنهادی را مستقیماً استفاده نکردیم و قبل از اضافه کردن آن‌ها، کد را بررسی و آزمون‌ها را اجرا کردیم.

روند استفاده به این صورت بود که ابتدا مسئله و محدودیت‌های همان بخش را مشخص می‌کردیم، سپس پیشنهاد تولیدشده را با ساختار موجود پروژه تطبیق می‌دادیم. بعد از اعمال تغییر، خروجی با اجرای برنامه، بررسی دستی و آزمون‌های خودکار

کنترل می‌شد. به این ترتیب هوش مصنوعی نقش ابزار کمکی داشت و تصمیم نهایی درباره طراحی، امنیت و صحت عملکرد همچنان بر عهده اعضای گروه بود.

نقطه قوت هوش مصنوعی این بود که برای کارهای تکراری و پیدا کردن خطاهای احتمالی سرعت خوبی داشت. از طرف دیگر، گاهی به دلیل نداشتن شناخت کامل از پروژه یا اجرا نکردن واقعی برنامه، پیشنهاد ناقص یا خیلی عمومی می‌داد. به همین دلیل بازبینی دستی و آزمون نهایی همچنان لازم بود.

۷. جمع‌بندی

در فاز اول رابط کاربری سامانه موسیقی را ساختیم و در فاز دوم آن را به بک‌اند، پایگاه داده و احراز هویت واقعی وصل کردیم. جدا کردن مسئولیت بخش‌ها، مشخص بودن رابطه مدل‌ها، آزمون‌های خودکار و اجرای داکری باعث شده‌اند ادامه دادن و توسعه پروژه در مراحل بعدی راحت‌تر باشد.

ساختار فعلی امکان اضافه کردن قابلیت‌هایی مانند روش‌های پرداخت جدید، گزارش‌های دقیق‌تر و توسعه پخش‌کننده را بدون بازنویسی کامل بخش‌های موجود فراهم می‌کند. همچنین جداسازی رابط کاربری از منطق سرور باعث می‌شود هر دو بخش بتوانند مستقل از یکدیگر بهبود پیدا کنند و در عین حال قرارداد مشترک خود را حفظ کنند.